

Ayudantía 7

Contenedores parte 3

Map y vector

Programación Avanzada | Sección: 03 | CIT 1010
Universidad Diego Portales

Contacto: isaias.vasquez1@mail.udp.cl Discord: **_iszzza**

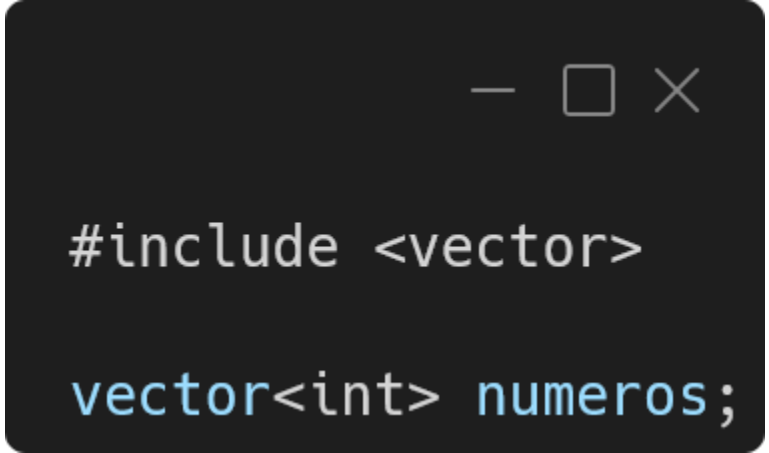
Github: github.com/Isza285/Programacion-Avanzada-2026-01

Sección 1

Vector

Vector

Un vector es similar a un arreglo, pero con la posibilidad de crecer de manera dinámica y no tiene tamaño fijo como un arreglo.

A dark-themed code editor window with standard window controls (minimize, maximize, close) in the top right corner. It contains two lines of C++ code: `#include <vector>` and `vector<int> numeros;`.

```
#include <vector>

vector<int> numeros;
```

Añadir y acceso

Para añadir elementos a un vector se utiliza el método `push_back(x)`, siendo `x` el valor ingresado al final del vector.

Para acceder a un elemento del vector se utiliza índices como en arreglos.

```
vector<int> numeros;  
  
numeros.push_back(1);  
numeros.push_back(2);  
numeros.push_back(3);  
  
cout << numeros[1];  
//Output: 2
```

Recorrer un vector

Se puede recorrer un vector utilizando un ciclo for de la siguiente manera:

```
for(int i=0; i<numeros.size(); i++){  
    cout << numeros[i] << endl;  
}
```

Métodos comunes

Método	Operación
<code>push_back(x)</code>	Ingresa un elemento x al final del vector.
<code>pop_back()</code>	Elimina el último elemento del vector.
<code>empty()</code>	Booleano sobre si esta vacío o no el vector.
<code>size()</code>	Retorna la cantidad de elementos del vector.

Sección 2

Map

Map

Un map guarda los datos mediante pares clave-valor.

En este caso la clave (índice) puede tomar valores diferentes a los índices de un arreglo.

Por ejemplo: la clave puede ser un string.

```
#include <map>

map<string,int> años;

años["Fabian"] = 23;
años["Carla"] = 20;

cout<<años["Fabian"];
//Output: 23
```


Como recorrer un map

Para recorrer un map, se utiliza un ciclo for, pero con una estructura diferente. En este caso se utiliza *auto*.

```
for(auto i : años){  
  
    cout<<i.first<<" : ";  
    cout<<i.second<<endl;  
  
} //Output:  
//Carla: 20  
//Fabian: 23
```

Nota: las claves se almacenan de manera ordenada, por lo que, Carla se imprime primero.

Métodos comunes

Método	Operación
[x]	Para insertar o acceder mediante la clave x.
erase(clave)	Elimina una clave.
.first	Obtiene la clave del map.
.second	Obtiene el valor del map.
size()	Retorna la cantidad de elementos.
empty()	Booleano sobre si esta vacio o no el map.

Sección 3

Ejercitación

Ejercicio 1

Crear un código que permita almacenar 5 notas de distintos alumnos en un `vector<int>`.

En el main:

- Ingresar 5 notas con `push_back(x)`.
- Recorrer el vector usando `for` e imprimir las notas.
- Imprimir la cantidad de elementos con `size()`.
- Eliminar la ultima nota con `pop_back()` y volver a imprimir el tamaño del vector.

Ejercicio 2

Se le solicita crear un `map<string,string>`, en la clave se almacenan nombres de personas, y en el valor su ciudad de residencia.

En el main:

- Añadir el nombre de 3 personas y sus ciudades.
- Se debe recorrer el map, mostrando sus claves y valores, utilizando los métodos de *first* y *second*.

Ejercicio 3

Crear un map de notas de diferentes asignaturas (map<string,int>).

En main:

- Añadir 3 asignaturas y 3 notas.
- Imprimir el map original.
- Eliminar una clave con erase().
- Imprimir nuevamente el map.

Ejercicio 4

Crear un vector<int> con 5 números y realizar lo siguiente:

- Solicitar un número al usuario.
- Recorrer el vector.
- Indicar si el numero existe dentro del vector.